Section 13

Section 12

Introduction to Linux and C++

Topics: Terminal Commands, compiling with g++, C++ basics
Lab 1: https://maryash.github.io/135/labs/lab_01.html

Ryan Vaz

Things you should know:

This course assumes you have some prior knowledge about programming. Most of the students are coming from CSCI 127 which covered the basics of C++:

- C++ basic syntax (variables, libraries, comments)
- Input / Output in C++
- Conditionals in C++ (if statements)
- C++ for-loops
- Basic math operations
- Common Unix/Linux terminal commands

C++ basic syntax (recap)

The following is a basic C++ program that prints “Hello World!”:

```cpp
#include <iostream>          // import iostream library (since we are using `cout`)
using namespace std;         // use standard namespace to avoid typing "std::"
int main() {                 // main function definition
    cout << "Hello World!"; // print "Hello World!" using cin
    return 0;                // end of
}
```

C++ variables(recap)

Variables in C++ are assigned a data type during initialization. Most common datatypes are: int, string, float, double, long etc. Usage

```cpp
#include <iostream>                 // import iostream library (since we are using `cout`)
using namespace std;                // use standard namespace to avoid typing "std::"
int main() {                        // main function definition
    int num = 9;                    // an integer num is set to 9
    double decimal = 4.2;           // an double decimal is set to 4.2
    string str = "Hello World!";    // an string str is set to "Hello World!"
}
```

C++ comments (recap)

Programmers explain and document their code using comments. Single-line comments start with two forward slash: // This is a single line comment
Multi-line comments start with “/*” and end with “*/”. Follow this template in the beginning of all code that you submit for this course

```
/*
Author: your name
Course: CSCI-136
Instructor: their name
Assignment: title, e.g., Lab1A
Here, briefly, at least in one or a few sentences describe what the program does.
*/
```

Input and Output in C++ (recap)

Basic input and output in C++ are part of the <iostream> library. Get inputs using `cin` and `cout` is used for output. For example:

```cpp
#include <iostream>                                   // required library for `cin` and `cout`
using namespace std;
int main() {
    int num;                                          // integer variable named `num`
    cin >> num;                                       // get the value of num from user input, e.g. 5
    cout << "You entered the number " << num << endl; // outputs "You entered the number 5"
                                                      // `endl` is used to go to the next line
    return 0;
}
```

if-statements in C++ (recap)

You should be familiar with the usage of if-statements or conditionals. The order of conditions matter. Here's an example of if-statements in C++:

```cpp
#include <iostream>
using namespace std;
int main(){
    if (num == 1){                  // if num is equal to 1
        cout<< "You entered 1" <<endl;
    }
    else if (num == 2){             // if num is equal to 2
        cout << "You entered 2" <<endl;
    }
    else{                           // if num is not 1 and not 2
        cout << "You entered a number that is not 1 or 2" << endl;
    }
}
```

for-loops in C++

If you want to repeat a task over and over again, use loops. The most common loop is the for-loop. Here's an example:

```
// i is an integer variable that changes in each iteration of the loop
// i (start = 0; stop = 3; step = 1)
// i will be 0, 1, 2 (notice it doesn't reach 3, since it starts from 0, repeats 3 times)
for (int i=0; i<3; i++) { // repeat 3 times
cout << "The value of i is " << i << endl; // outputs value of i
}
/* Output: The value of i is 0
The value of i is 1
The value of i is 2 */
```

The modulo operator (%)

The modulo operator (%) computes the remainder of a division operation. For example, 37 % 10 returns 7, because this is the remainder of 37 when divided by 10. This can be very useful for different tasks. Here's an example:

```
int num;
cin << num;                          // get input from user
if (num % 2 == 0){                   // if num divided by 2 has remainder of 0
    cout << "Even number" << endl;   // it is an even number
}
else {
    cout << "Odd number" << endl;    // otherwise it is an odd number
}
```

Linux Commands

Command	Description
pwd	print the current working directory
ls	list files in the current directory
ls path/to/a/directory	list files in the directory
cd path/to/a/directory	change directory
cd ..	go to the parent directory (one level up)
mkdir newdirectoryname	create new directory
cp file1 file2	copy file1 and call the copy file2
mv file1 file2	rename (move) file1 to file2
rmdir directoryname	remove empty directory
rm file	remove file

Compiling code with g++ in the terminal

In Linux/Unix terminals, `g++` allows compilation of C++ code. Let's say that we have some code written in a file called `code.cpp`. If we compile this using `g++`, `g++` will create an executable file.

We can configure the name of this file during compilation:

- Compile with default executable name: `g++ code.cpp`
- Compile with custom executable name: `g++ -o myprog code.cpp`

Running the executable:

- Default executable: `./a.out`
- Custom executable: `./myprog`

Which text editor to use?

There are different IDE and text editors out there that can be used with C++. Most common ones are: VS code, Vim, eclipse, sublime text, gedit etc.

Try out some of them and see which one works for you. I personally recommend using VS code since it works with different programming languages and supports different plugins.

Check the course webpage daily for updates and materials. Each homework and lab prepares you for the quizzes and exams. Keep up with the deadlines and due dates for all assignments and you will be prepared for the exams and quizzes.

Course Webpage: https://tong-yee.github.io/135/2025_spring.html

Good Coding Practices

Computer Science is a very competitive field. What separates a good programmer from the competition is how readable their code is. This includes being consistent in syntax, descriptive variable names, proper documentation and comments etc.

While your code might compile and run, it maybe difficult to understand for other programmers. Being consistent will help both you and the UTA that is trying to help you debug your code. Try to follow this style-guide:

https://maryash.github.io/135/worked_examples/style_guide.html